

intel_mgmt_mon.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/host_if/intel_mgmt/intel_mgmt_mon.vr

```
/*
 * Author:      Fabian Delguste
 * File:        $RCSfile: intel_mgmt_mon.vr,v $
 * Revision:    $Revision: 1.10 $
 * Date:        $Date: 2004/01/19 15:37:11 $
 *
 * *****
 *
 * Basic Transaction Monitoring Module aimed at
 * creating monitor() transaction.
 *
 * This transaction constantly watches the intel bus,
 * pushing the transaction onto a rvm_channel each time
 * a {READ, WRITE} cycle terminates.
 *
 * *****
 */
typedef class intel_mgmt_mon;
class intel_mgmt_mon_callbacks extends rvm_xactor_callbacks {
    // Called after a new transaction has been observed
    virtual task post_mon(intel_mgmt_mon  mon,
                          intel_mgmt_trans tr,
                          var bit         drop)

    {}
}

class intel_mgmt_mon extends rvm_xactor {
    // Factory instance for transaction descriptors
    intel_mgmt_trans allocated_tr;
    // Interface to Intel Bus HDL signals
    local intel_mgmt_port iport;
    // Output channel of observed transactions
    intel_mgmt_trans_channel out_chan;

    // Are transactions put into the output channel?
    local bit is_sink;
    // Constructor
```

```

task new(string                instance,
          integer              stream_id,
          intel_mgmt_port      iport,
          intel_mgmt_trans_channel out_chan = null,
          bit                  is_sink = 0);

// From rvm_xactor
virtual task reset_xactor(integer rst_type = 0);
protected virtual task main_t();
// Monitors the bus and returns whenever a new transaction
// has been observed
local task monitor_t(var intel_mgmt_trans tr);
}

task intel_mgmt_mon::new(string                instance,
                          integer              stream_id,
                          intel_mgmt_port      iport,
                          intel_mgmt_trans_channel out_chan = null,
                          bit                  is_sink = 0)
{
    super.new("Intel Mgmt Mon", instance, stream_id);
    this.allocated_tr = new;

    this.iport = iport;

    if (out_chan == null) {
        out_chan = new({this.log.get_name(), " Output Channel"}, instance);
    }
    this.out_chan = out_chan;
    this.is_sink = is_sink;
    // Make sure output channel will never block
    out_chan.reconfigure(65535);
}

task intel_mgmt_mon::reset_xactor(integer rst_type = 0)
{
    super.reset_xactor(rst_type);
    this.out_chan.flush();
}

task intel_mgmt_mon::main_t()
{
    fork
        super.main_t();
}

```

```

join none
while (1) {
    intel_mgmt_trans tr;
    bit                drop = is_sink;
    super.wait_if_stopped_t();
    cast_assign(tr, this.allocated_tr.allocate());
    this.monitor_t(tr);
    if (this.log.start_msg(this.log.DEBUG_TYP, this.log.TRACE_SEV)) {
        if (this.log.text("Observed transaction...")) {
            void = this.log.text();
            tr.display("  ");
        }
        this.log.end_msg();
    }
    rvm_00_callback(intel_mgmt_mon_callbacks,
                    post_mon(this, tr, drop));

    if (!drop) {
        if (this.log.start_msg(this.log.DEBUG_TYP, this.log.DEBUG_SEV)) {
            if (this.log.text("Forwarding transaction...")) {
                void = this.log.text();
                tr.display("  ");
            }
            this.log.end_msg();
        }

        fork
            rvm_warning(this.log, "Output channel is full. Monitoring temporarily suspended"
            join none
            this.out_chan.put_t(tr);
            terminate;
    }
}

task intel_mgmt_mon::monitor_t(var intel_mgmt_trans received_mt)
{
    bit Sel;
    bit Rd_nWr;

    // Wait for the device to be selected
    Sel = this.iport.$Sel async;
    if(Sel != 0) {
        @(negedge this.iport.$Sel async);
    }
}

```

```

}
// Wait for a Rd or Wr cycle
fork
{@(negedge this.iport.$Rd async);}
{@(negedge this.iport.$Wr async);}
join any
terminate;
// Read cycle
Rd_nWr = !this.iport.$Rd async;

if(Rd_nWr) {
    // Wait for the Rd latch enable
    @(posedge this.iport.$Rd async);
    // Store current transaction parameters
    received_mt.kind = intel_mgmt_trans::READ;
    received_mt.data = this.iport.$Data async;
    received_mt.addr = this.iport.$Addr async;
}
// Write cycle
else {
    // Wait for the Wr latch enable
    @(posedge this.iport.$Wr async);
    // Store current transaction parameters
    received_mt.kind = intel_mgmt_trans::WRITE;
    received_mt.data = this.iport.$Data async;
    received_mt.addr = this.iport.$Addr async;
}
}

```